



Sardar Patel Institute of Technology

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058, India

(Autonomous College Affiliated to University of Mumbai)

Mid Semester Examination

August – 2017

Max. Marks: 30

Class: T.E.I.T.

Course Code: ITC502

Name of the Course: Operating Systems

Duration: 90 minutes

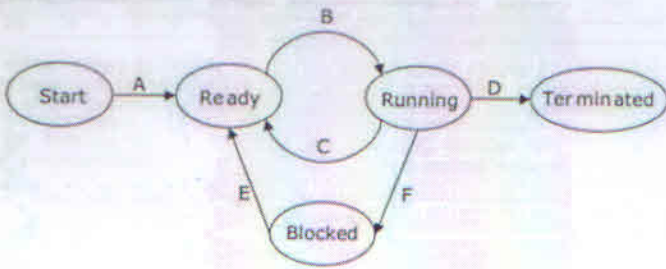
Semester: V

Branch: IT

Instruction:

- (1) All questions are compulsory
- (2) Draw neat diagrams
- (3) Assume suitable data if necessary

Q No.		Max. Marks	CO
Q.1 (a)	In a multiprogramming and time sharing environment, several users share the system simultaneously. What are the two security problems results in this situation?	1	CO1
Q.1 (b)	Mention any 2 main objectives of Operating Systems?	1	CO1
Q.1 (c)	Explain the need of multiprogramming in Operating Systems with example?	2	CO1
Q.1 (d)	What does fork() do? hence what will be the output of the following program. <pre>#include<stdio.h> #include<sys/types.h> #include<unistd.h> int main() { fork (); printf(" Hello _world!\n"); return 0; }</pre>	2	CO1

Q.2 (a)	Consider three processes, all arriving at time zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again. The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst. Assume that all I/O operations can be overlapped as much as possible. For what percentage of time does the CPU was busy?. List three types of scheduler along with their frequency and the work they do. OR In the following process state transition diagram for a uniprocessor system, assume that there are always some processes in the ready state:  <pre> graph LR Start([Start]) -- A --> Ready([Ready]) Ready -- B --> Running([Running]) Running -- C --> Ready Running -- D --> Terminated([Terminated]) Running -- F --> Blocked([Blocked]) Blocked -- E --> Ready </pre> Now consider the following statements: - If a process makes a transition D, it would result in another process making transition A immediately. - A process P2 in blocked state can make transition E while another process P1 is in running state. - The OS uses preemptive scheduling. - The OS uses non-preemptive scheduling Which of the above statements are TRUE? Justify your answer. - I and II - I and III - II and III - II and IV .	5	CO2
Q.2 (b)	Differentiate between User Level Thread and Kernel Level Thread.	4	CO2
Q.2 (c)	c) Write down two advantage of Shortest Job First Algorithm. $\alpha = 0.5$, $\tau_1 = 10$, actual burst times $(t_1, t_2, t_3, t_4) = (4, 8, 6, 7)$. Calculate τ_5.	3	CO2

Q.3 (a)

Three concurrent process X,Y,Z access and updates shared variable. They uses 4 semaphores a,b,c,d such that X uses (a, b, c) , Y uses (b, c, d) and Z uses (c, d, a). Which of the following is/are deadlock order.

Sr. No	X	Y	Z
1	p(a), p(b), p(c)	p(b), p(c), p(d)	p(c), p(d), p(a)
2	p(b), p(a), p(c)	p(b), p(c), p(d)	p(a), p(c), p(d)
3	p(b), p(a), p(c)	p(c), p(b), p(d)	p(a), p(c), p(d)
4	p(a), p(b), p(c)	p(c), p(b), p(d)	p(c), p(d), p(a)

Justify your answer.

OR

Two processes, P1 and P2, need to access a critical section of code. Consider the following synchronization construct used by the processes:

P1	P2
<pre>while (true) { wants1 = true; while (wants2 == true); /* Critical Section */ wants1=false; } /* Remainder section */</pre>	<pre>while (true) { wants2 = true; while (wants1==true); /* Critical Section */ wants2 = false; } /* Remainder section */</pre>

Here, wants1 and wants2 are shared variables, which are initialized to false. Find whether above construct satisfies Mutual Exclusion but could not prevent deadlock or vice versa. Justify your answer. What is critical section.? What is race condition. Explain bounded wait and progress with example.

Q.3 (b)

Considering a system with five processes P0 through P4 and three resources types A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t_0 following snapshot of the system has been taken. What will be the content of the Need matrix? Is the system in safe state? If Yes, then what is the safe sequence?

Process	Allocation	Max	Available
	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	3 3 2
P ₁	2 0 0	3 2 2	
P ₂	3 0 2	9 0 2	
P ₃	2 1 1	2 2 2	
P ₄	0 0 2	4 3 3	

Q.3 (c)

What are the necessary conditions for deadlock?

5

CO3

5

CO3

2

CO3